

DSST389: Exam III — Reference Sheet

This sheet is attached to your exam. You do not need to memorize any of it.

Building a network. A network is a set of **nodes** and a set of **edges** joining pairs of them. It is recorded as an **edge list**: one row per edge, with the endpoints in the columns `doc_id` and `doc_id2`.

```
node, edge, G = DSNetwork.process(edges, directed=False)
```

`node` has one row per node: `id`, the layout coordinates `x` and `y`, and the metrics below.

Components. A **component** is a set of nodes all reachable from one another by following edges. The `component` column numbers them in **descending order of size**, so component 1 is always the largest, and `component_size` gives the size of each.

Centrality. Distances, and the scores built on them, are computed **separately within each component**. Filter to `component == 1` before comparing scores across nodes.

- **degree** — the number of neighbors a node has. Purely local.
- **close** — *closeness centrality*. Take the shortest-path distance from the node to every other node in its component, and add up the **reciprocals**. A neighbor contributes 1, a node four hops away contributes only $1/4$, so the score rewards being near *everything*.
- **between** — *betweenness centrality*. For every pair of other nodes, consider the shortest paths between them; count what fraction of them run *through* this node, and add that up over all pairs. High for **bridges**: nodes that are not popular but are unavoidable.
- **cluster** — a community label, assigned by an algorithm that tries to put more edges inside groups than between them. It is an output, not a property: a node on the boundary between two groups may be assigned arbitrarily.

Co-citation. An edge between two pages whenever some *third* page cites **both**. Always undirected. The `count` column holds the number of citing pages a pair shares.

Images as arrays. `cv2.imread(path)` returns a NumPy array of shape (**height, width, channels**) with values from 0 to 255. `img.mean()` averages over **every number**, channels included: a 2×2 color image has 12 of them, not 4.

- **OpenCV stores channels in BGR order, not RGB.** `[255, 0, 0]` is *blue* and `[0, 0, 255]` is *red*. `[0,0,0]` is black and `[255,255,255]` is white.

Images as vectors.

- `SigLIPEmbedder()` — embeds **images and text into the same space**, with `.embed_image(path)` and `.embed_text(string)`. The vectors are unit length, so `dot_product(a, b)` is exactly the cosine similarity of Chapter 15: 1 means identical, 0 means orthogonal, and the angle distance is $\arccos$ of it.
- **Zero-shot classification:** embed each category description as text, and give each image the label whose text vector it has the largest dot product with. No training and no labeled examples — an *argmax* over the candidate list you supplied.